

LAB 6 – Build application with Json Server

Rules:

- The web browser must access the project **via localhost only**
- Any access via external or online URLs will result in a **score of zero**
- Internet access is only allowed to **download required packages** (e.g., npm packages)
- If the project has any errors when running, it will automatically receive a **score of zero**
- Students are allowed to use **any version of React** to complete the exam
- Students are allowed to use **any materials or resources already** saved on their computers
- All required images and the **database file (db.json)** are included in the folders provided
- Students can use **Bootstrap** for designing the user interface

Instructions

Step 1:

1. Download the provided materials package
2. Unzip the package. It includes the following folders
 - a. The `api` folder contains the `db.json` file
 - b. The `images` folder contains Shirt images used in the project. Copy these images into the `public` folder of your project

Step 2: Create a React project named **lab6_<memberCode>** (lowercase). (ex: lab6_sangnqce150621). Inside the project folder, create a new folder named `api`, and copy the `db.json` file into the `api` folder

Step 3: Set Up and Start JSON Server

1. Navigate to the **lab6_<memberCode>** folder using terminal
2. Run the following command to install JSON Server

```
npm install json-server
```
3. Run command to start the server on port 5000

```
npx json-server --watch api/db.json --port 5000
```

After the JSON Server started successfully on port 5000, the API data could be accessed at the following endpoints:

```
http://localhost:5000/home
http://localhost:5000/shirts
http://localhost:5000/contact
```

Note: Keep this terminal tab open while working on the project. Do not close it, as it is required to serve the data

Step 4: Take the exam

1. Carefully review the data structure in `db.json` to understand it clearly.
2. Open another terminal window to run the command: `npm start`
3. Create a **components** folder and add the components that implement the requirements specified in the exam.

Step 5: Submit the exam

1. Before submitting the solution, delete the **[node_modules]** folder in your exam to reduce the space.

Click the Browse button and select the **lab6_<memberCode>** folder to submit your exam on PEA Client software

Request 1: Create Homepage [20 points]

- Route: /
- Create a homepage that displays content from the endpoint
GET <http://localhost:5000/home>
- Show the *title* and *content* fields (HTML content must be rendered properly)
- The UI should be displayed (Figure 1)

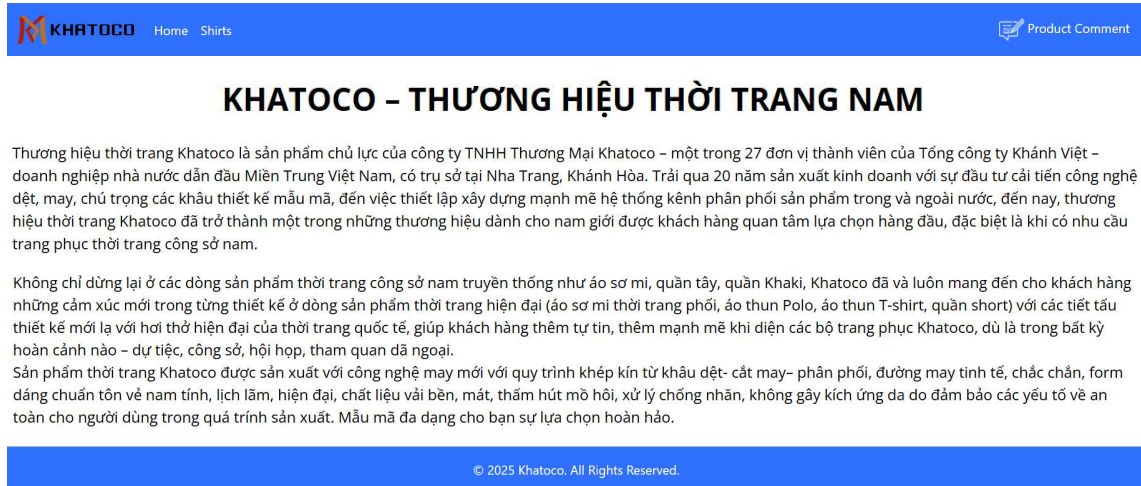


Figure 1: Home page of Khatoco

Request 2: Shirts List Page [25 points]

- Route: /shirts
- Fetch and display all cars from the API: GET <http://localhost:5000/shirts>
- Display each Shirt with name, new, price, and image (Figure 2)
- Each Shirt should have a View Detail button linking to /shirts/:id

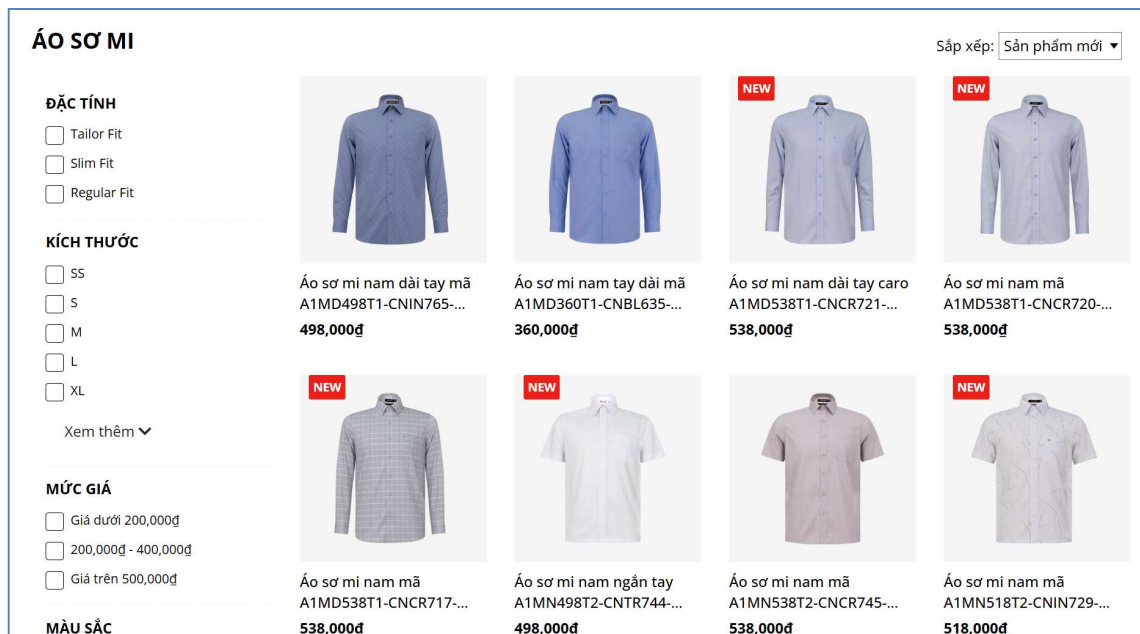


Figure 2: List of shirts page

Request 3: Shirts Detail Page [20 points]

- Route: /shirts/:id
- When clicking View Detail, navigate to the detail page

- Shirt id data must be passed and fetch and display a Shirt from the API:
GET <http://localhost:5000/shirts/:id>
- Display all details of the selected Shirt (Figure 3)

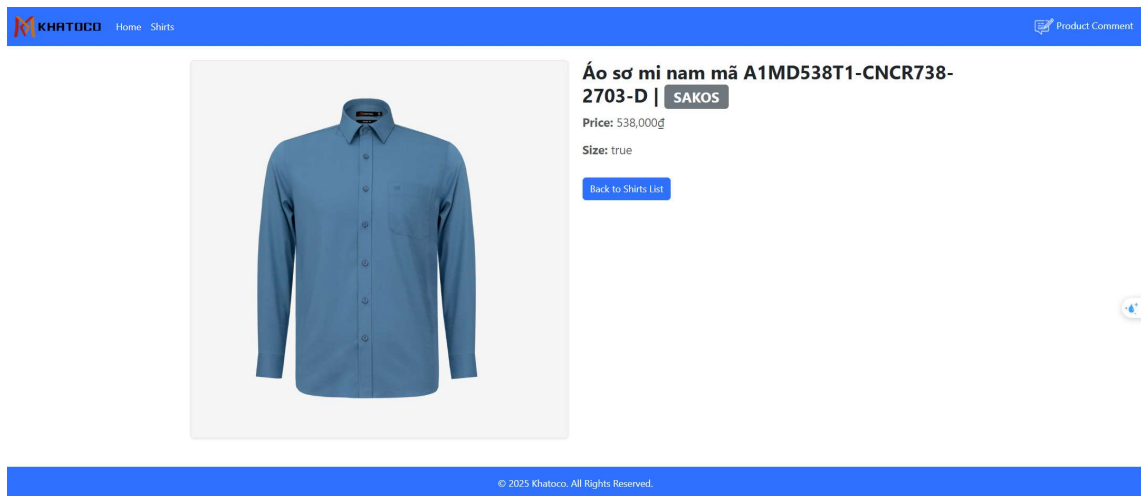
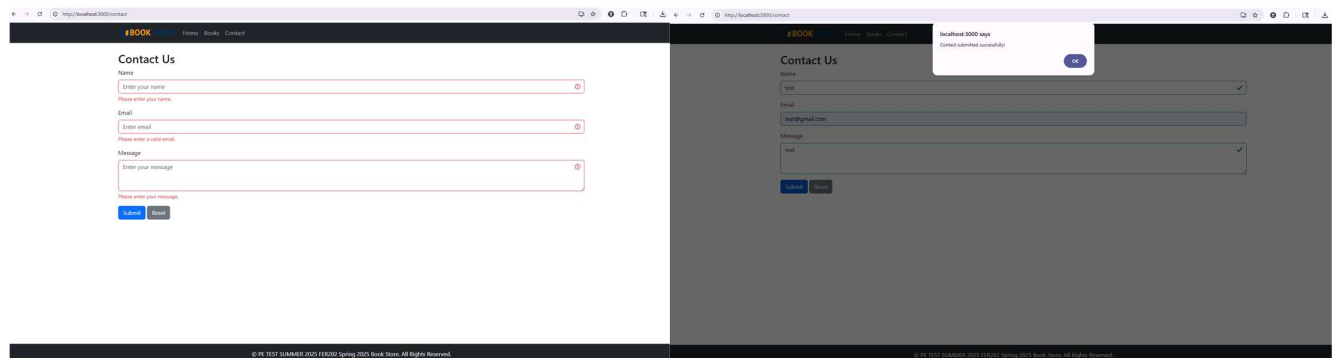


Figure 3: Detail of Shirt page

Request 4: Contact Form Page [25 points]

- Route: /contact
- Create a form with the following fields (Figure 4)
 - name (text)
 - email (email)
 - message (textarea)
- Include 2 buttons Submit and Reset and use HTML5 form validation
- On submit, send the data to: POST <http://localhost:5000/contact>
- Reset button clears the form



a) Contact validation failed

b) Contact form submitted successfully

Figure 4: Contact page

Request 5: Add the logo (logo.png) to the header from the `images` folder. All UI elements must be responsive [10 points]

Scoring Criteria Table

#	Criteria	Points
Request 1		20
1.1	Correctly sets up route /	5
1.2	Successfully fetches post data from GET http://localhost:5000/home	10
1.3	Uses Bootstrap styling to format the homepage layout like Figure 1	5
Request 2		25
2.1	Correctly sets up route /cars	5
2.2	Successfully fetches Shirt data from GET http://localhost:5000/cars	10
2.3	Uses Bootstrap layout and matches the design of Figure 2	10
Request 3		20
3.1	Correctly sets up dynamic route /cars/:id	5
3.2	Uses the ID to fetch single Shirt data from GET http://localhost:5000/cars/:id	10
3.3	Detail page layout follows Figure 3 and uses Bootstrap styling	5
Request 4		25
4.1	Correctly sets up route /contact	5
4.2	On valid submit, sends form data using POST to http://localhost:5000/contacts	10
4.3	Uses Bootstrap styling and matches Figure 4	10
Request 5		10
5.1	Header and footer Styled correctly (including logo and text)	5
5.2	Responsive UI with basic mobile support	5

----- THE END -----